

This block is where we will write all our redirection rules. For testing purposes, start any of the application servers that you may have on any of the internal servers. Let's assume that our public IP address is a.b.c.d:6500 and let's take two application servers, one running Tomcat on 10.0.0.1 on Port 8080 and another running the *nodejs* server on 10.0.0.2 on Port 3000. Now, to add rules for these two application servers, simply modify the above code block:

```
<IfModule mod_rewrite.c>
    RewriteEngine On
#Tomcat Rewriting Rules
    #Matching pattern for sampleWebAppOne, to redirect to
    local tomcat server running on 8080 port
    RewriteRule ^/sampleWebAppOne /(.*)$
    http://10.0.0.1:8080/sampleWebAppOne /$1 [L,P]
#Nodejs Rewriting Rules
    #Matching pattern for sampleWebAppTwo, to redirect to local
    nodejs server running on 3000 port
    RewriteRule ^/sampleWebAppTwo /(.*)$
    http://10.0.0.2:3000/sampleWebAppTwo/$1 [L,P]
</IfModule>
```

mod_rewrite is nothing but *regex* magic. Here, we are matching request patterns and redirecting those requests to other local internal servers based on the matched patterns. For the *sampleWebAppOne*, we send a request like the following:

```
http://a.b.c.d:6500/sampleWebAppOne/someResource
```

So this request gets translated based on the pattern that we have written above. The *regex* is looking for a pattern that matches *'sampleWebAppOne'* and any number of characters following it. Next, if the pattern matches, this request is redirected to the internal server. Here, whatever falls into the category of 'any number of characters following it' gets passed to *'\$1'*. So, the above request gets translated to:

```
http://10.0.0.1:8080/sampleWebAppOne/someResource
```

So is the case for the *nodejs* server rule. Next, there are specific flags that can be used with these rules. In our rules, we have used the flags [L,P]. The meaning of the L flag is that if the rule matches, no other rules should be processed. The P flag redirects the request to *mod_proxy*. This is how we achieve the reverse proxy configuration. Using this, we can map the remote content into the *namespace* of the local server without exposing the local server to the public.

Finally, to test these, first restart the Apache server:

```
/etc/init.d/httpd restart
```

```
## OR ##
service httpd restart
```

If everything is fine, the server will restart gracefully without any errors. To confirm if the server is up and running, type the following command:

```
netstat -tulnp | grep :portNumber
```

Next, try accessing any of the resources on the Tomcat server or the *nodejs* server using the requests with the public IP and the respective port. If the rewriting was successful, you will be able to access those resources without seeing a change in the IP in the address bar. This means you have achieved reverse proxy configuration along with URL rewriting to cater to requests from a single public IP, without exposing the internal servers. **END** 

By: Manit Singh Kalsi

The author works as a mobile evangelist in the Mobility Center of Excellence at Genpact Headstrong Capital Markets. He is a Java and JavaScript developer who enjoys exploring new technologies and frameworks. When not coding, he is either strumming his guitar or playing video games. Follow him @manitsinghkalsi




2 Simple Ways To Stay Up-to-date With India's Electronics Industry

Want Latest News?
Check out
EFYTimes.com



Want Industry Trends?
Check out
Electronics2b.com



For any queries, please contact our team at efyindia@efyindia.com OR +91-11-26810601